

Pokhara University
Gandaki College of Engineering and Science
Lamachaur, Pokhara-16, Kaski



**Lab 3: Implementation of Software Testing – Test Case
Generation, JUnit, Jest, Selenium, and Load Testing**

Submitted By:

Ankit karmacharya

Roll no. 10

Submitted To:

Er. Rajendra Bahadur Thapa

Lab Report

Objective

To implement software testing techniques using test case generation, JUnit, Jest, Selenium, and load testing tools to verify software functionality and performance.

Tools Required

- Java (JDK)
- Node.js
- JUnit
- Jest
- Selenium WebDriver
- Apache JMeter
- Visual Studio Code / IntelliJ IDEA / Eclipse

Theory

Software testing ensures that an application functions correctly and meets user requirements. JUnit is used for unit testing Java applications, Jest for JavaScript testing, Selenium for web application automation, and Apache JMeter for evaluating application performance under different workloads.

Procedure

1. Prepare the application for testing.
2. Generate test cases for the required functionalities.
3. Execute unit tests using JUnit.
4. Execute JavaScript tests using Jest.
5. Automate browser testing using Selenium WebDriver.
6. Perform load testing using Apache JMeter.
7. Analyze and verify the test results.

Sample Test Cases

- Verify login with valid credentials.
- Verify login with invalid credentials.
- Check validation for empty input fields.
- Verify logout functionality.
- Confirm expected output after execution.

Coding Section

JUnit Example

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest {
    @Test
    public void testAddition() {
        assertEquals(10, 5 + 5);
    }
}
```

Jest Example

```
test("adds two numbers", () => {
    expect(5 + 5).toBe(10);
});
```

Selenium Example

```
WebDriver driver = new ChromeDriver();
driver.get("https://example.com");
System.out.println(driver.getTitle());
driver.quit();
```

Load Testing

Use Apache JMeter to create virtual users, send requests to the application, and analyze response time, throughput, and error rate.

Result

Successfully generated test cases, performed unit testing with JUnit and Jest, automated browser testing using Selenium, and evaluated application performance through load testing.

Conclusion

The experiment demonstrated the implementation of software testing techniques using JUnit, Jest, Selenium, and Apache JMeter. These tools help ensure software correctness, reliability, and performance.